

Cyber AI Platform Architecture Baseline — Views & Diagrams

C4 models · Deployment · Logical planes · Data flows · Component views ·
Sequence diagrams

Agentic Graph RAG · Governed Execution Harness · Multi-Agent Setup

Status	Baseline for implementation (v1.4 — workflow-as-a-graph view + framework decision + MVP minimal scope added; supersedes v1.3)
Date	July 23, 2026
Derived from	Consolidated Design Specification v1.0 + enterprise alignment updates
Enterprise stack	OpenShift on-prem · Microsoft Entra ID · Enterprise LLM Gateway · Enterprise Agent Registry · Enterprise Observability · Neo4j · RAG-as-a-Service · Postgres
Out of scope	Jira draft/creation write path (deferred)

CONTENTS

1. Enterprise Architecture Views

Enterprise stack mapping · C4 Level 1 (context) · C4 Level 2 (containers) · C4 Level 3 (tool-gateway components) · Deployment on OpenShift · Logical six-plane view · End-to-end sequence

2. Data Flow Diagrams

DFD-0 platform context · interactive request flow · knowledge ingestion flow · governed NL2Query flow · evidence & audit flow · CyberShim findings flow

3. Component & Sequence Views (Build vs Reuse)

Container view with enterprise systems of record · component internals (tool-gateway, agent-runtime, registry adapter, audit pipeline, CyberShim boundary) · sequences: end-to-end, governed NL2Query, CyberShim fix flow

4. Use-Case Sequences

Vulnerability chain analysis · knowledge Q&A (Graph RAG) · ask-the-data (NL2SQL/Cypher) ·

investigation/alert triage · threat modeling · control mapping · composed multi-agent review

5. **Workflow Expansion — Agent Bindings**

Activity-by-activity flowcharts with per-step agent/skill bindings · the workflow as a DAG (static edges vs the model's in-node sandbox) · recorded framework decision: Temporal-only orchestration, LangGraph as constrained in-activity library, ADK rejected

6. **MVP — Minimal Build**

10 services + 1 library · 1 agent, 1 workflow, 9 skills + 1 scorer, 8 read tools · templated-query guard posture · minimal infra · deferred list with seams · definition of done

Companion project documents: Architecture Overview (blocks & invariants) · Build-vs-Reuse Alignment · Registry Catalog (agents/skills/tools) · Production-Scale Architecture ToC.

The LLM never touches a database, a graph, or an enterprise system. It produces text; deterministic code validates, executes, and records.

Cyber AI Platform — Enterprise Architecture Views

C4 (context → container → component), deployment on **OpenShift on-prem**, logical planes, and the end-to-end sequence — adapted to your enterprise landscape: **Microsoft Entra ID** for auth, the **enterprise LLM Gateway API** for all model access, **Neo4j on-prem** for the graph, **enterprise RAG-as-a-Service** for vector retrieval, and **Postgres on-prem** for relational state.

0 · How the design maps onto your enterprise stack

One decision per constraint — these drive every diagram below.

YOU HAVE

HOW THE PLATFORM USES IT

Microsoft Entra ID

Single IdP. Users sign in via OIDC/PKCE; **context-api validates the Entra access token (JWKS)** and exchanges it for the platform's short-lived signed context token. Entra **groups → platform roles/entitlements** (analyst, approver). Services authenticate to each other with mTLS/workload identity inside OpenShift — user Entra tokens are **never replayed downstream**.

Enterprise LLM Gateway API

The platform's **model-gateway-service becomes a thin adapter** in front of it: it keeps prompt-registry enforcement, per-tenant/per-execution token budgets, and cost events — but delegates all inference & embeddings to the enterprise gateway. No service calls the LLM gateway directly except model-gateway-service.

Neo4j on-prem

Hosts the **one graph**: operational topology + knowledge edges on the canonical entity spine. Reached only via tool-gateway → cyber-domain-api, using a **read-only service account**; guards clamp depth and inject tenant scoping before any Cypher executes.

Enterprise RAG service

Provides the **vector + lexical channels** of hybrid retrieval (ingestion, chunking, embedding, ANN as a service). The platform adds what it can't: **graph expansion on Neo4j, entitlement pass-through (Entra groups as pre-query filters), deterministic fusion/rerank, and ev:// evidence packaging**. If the RAG service can't filter by entitlement pre-query, that's a hard gap to close — post-hoc filtering violates invariant I9.

Postgres on-prem

System-of-record for platform state: tenants, sessions, conversations, registry definitions, policies, execution metadata, approvals. **Schema-per-service** (or DB-per-service), API-only cross-access.

OpenShift on-prem

All 13 platform services + Temporal run as pods. **Trust zones A-F become namespaces/projects; the plane invariants become NetworkPolicies** — e.g.

"only tool-gateway pods can reach Neo4j/RAG/warehouse" is enforced by the cluster, not by convention. Service mesh (mTLS) for east-west traffic.

Kafka (on-prem)

Event backbone: **audit.events · tool.calls · policy.decisions · workflow.lifecycle · approvals · cost.usage · knowledge.ingest · query.audit** — CloudEvents envelopes, schema-registry governed, buffered fire-and-forget so audit never blocks execution. Feeds the immutable audit-of-record sink and the ingestion plane.

Redis (on-prem)

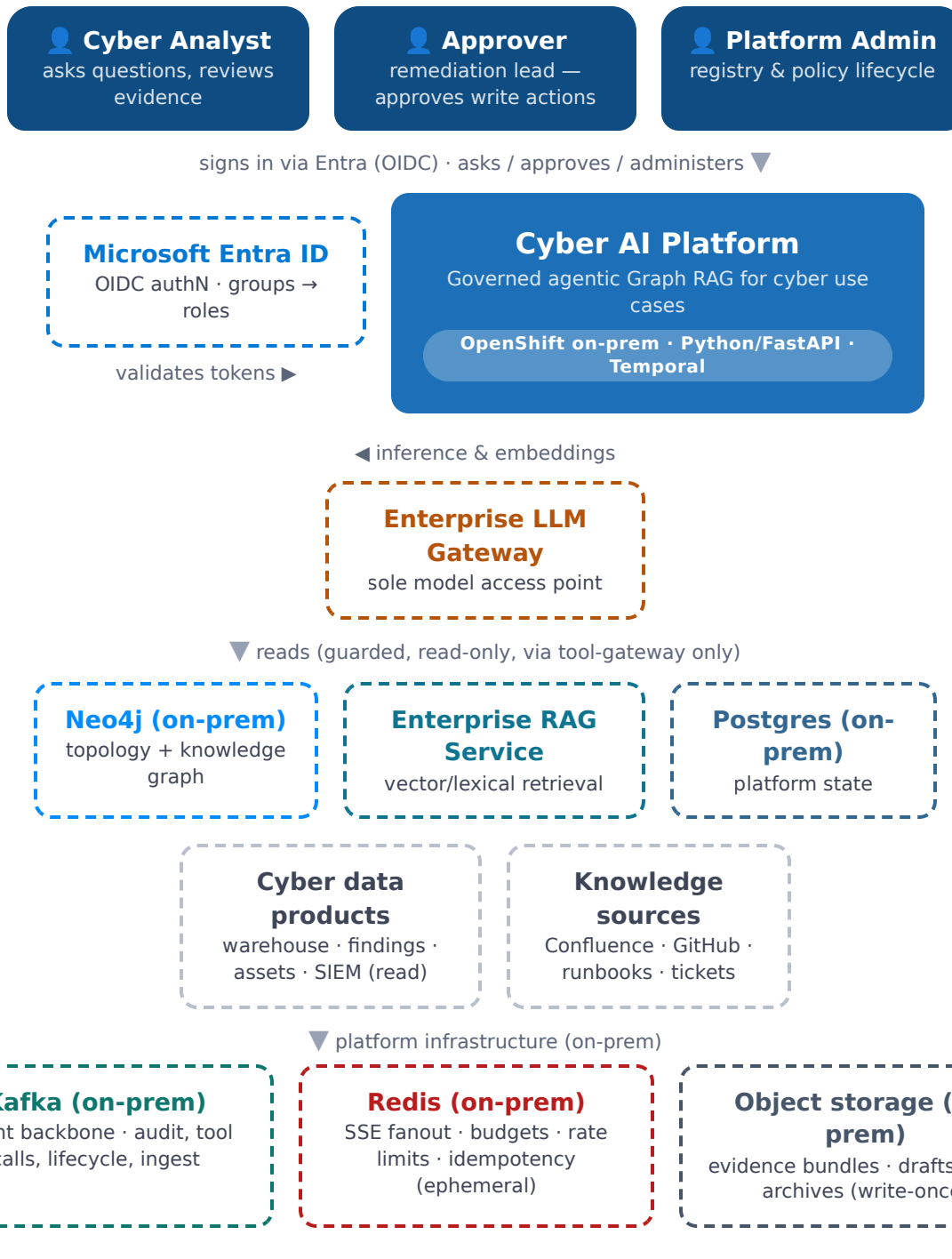
Ephemeral speed layer only — **SSE fanout streams, progress cache, rate limits, per-execution token budgets, idempotency keys, policy read-decision TTL cache**. Nothing durable lives here; loss of Redis degrades streaming and budget checks, never correctness of record.

Object storage (on-prem, S3-compatible)

System of record for **evidence bundles (ev://resolution), draft payloads, audit-of-record archives**. Write-once discipline; superseded artifacts tombstoned so historical evidence refs always resolve.

1 · C4 Level 1 — System Context

Who uses the platform and what it talks to. The platform is one system boundary; everything else is enterprise-shared.



2 · C4 Level 2 — Containers

The services inside the platform boundary, grouped by trust zone. Each box is a deployable container (pod). Arrows flow strictly top-down; red is the only zone with routes to enterprise data.

ZONE A · EXPERIENCE

chat-api

intake · SSE streaming · status — validates Entra token at the edge

ZONE B · CONTROL (GOVERNANCE)

context-api

Entra token → signed context
token · session · memory

orchestrator

intent → approved workflow

registry-api

versioned defs · semantic models
· contracts

policy-api

allow/deny · constraints ·
obligations

ZONE C · EXECUTION HARNESS

workflow-runtime

Temporal client · durable
executions · signals

Temporal cluster

server + workers · history in
Postgres

agent-runtime

skills · budgets · stateless
reasoning

model-gateway

adapter → Enterprise LLM
Gateway · prompt registry ·
budgets

ZONE D · GUARDED EGRESS — SOLE PATH TO ENTERPRISE DATA

tool-gateway

schema → policy → guard chain (AST rewrite, clamps,
masking) → adapters → audit

cyber-domain-api

findings · assets · graph paths · knowledge retrieve ·
evidence

ZONE F · ASSURANCE (CROSS-CUTTING)

verification-response

quality gate · NO egress

audit-observability

Kafka consumer of record · traces
· cost

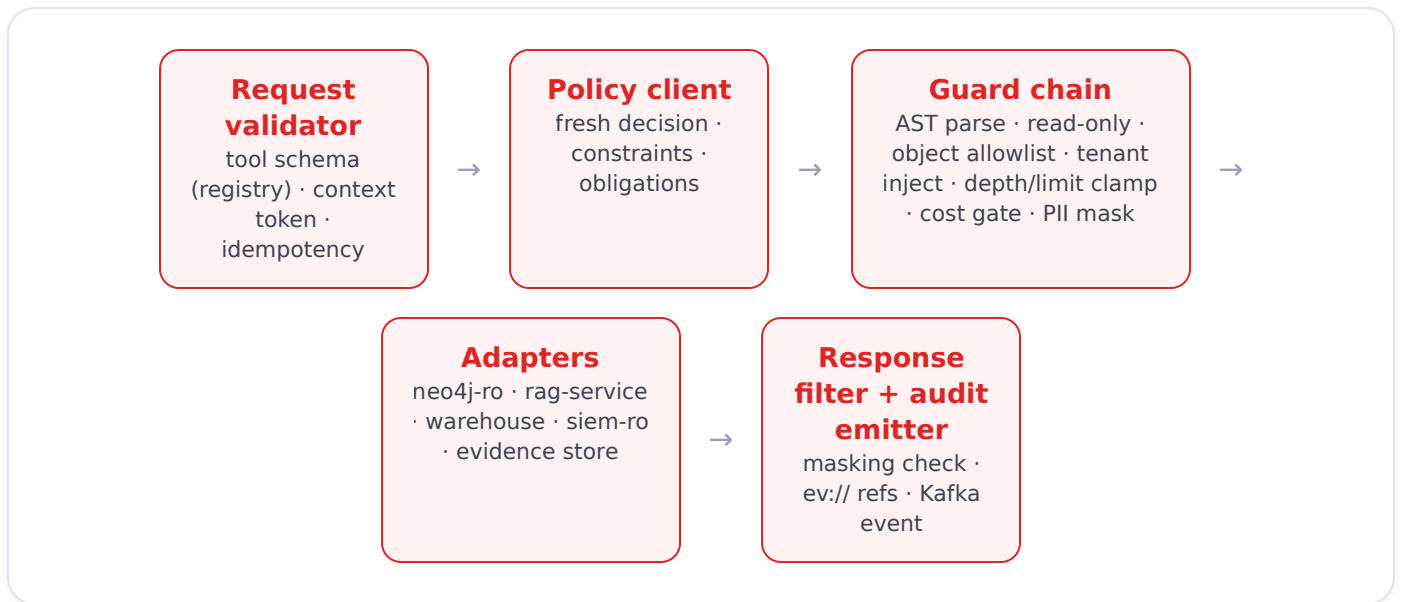
notification

approvals · Entra-group approver
checks

backing stores (API-only access): **Postgres** · **Neo4j** · **RAG service indexes** · Redis (SSE fanout, budgets) · Kafka (events) · OpenSearch (audit/evidence search) · object store (evidence bundles)

3 · C4 Level 3 — Components inside tool-gateway (the critical container)

Zoomed into the one container everything depends on. Every call passes left to right; no bypass path exists.



4 · Deployment Diagram — OpenShift on-prem

Trust zones become namespaces; invariants become NetworkPolicies. Data tier and enterprise shared services sit outside the cluster.



ON-PREM DATA CENTER

○ **OpenShift cluster** service mesh (mTLS east-west) · OpenShift Route/Ingress + WAF at the edge

namespace: cyber-ai-experience (Zone A)

chat-api ×3

redis clients → on-prem Redis (SSE fanout · budgets)

namespace: cyber-ai-control (Zone B)

context-api ×2

orchestrator ×2

registry-api ×2

policy-api ×2

namespace: cyber-ai-execution (Zone C)

workflow-runtime ×2

temporal-server

temporal-workers ×N

agent-runtime ×3

model-gateway ×2

namespace: cyber-ai-egress (Zone D — only namespace with egress to data tier)

tool-gateway ×3

cyber-domain-api ×2

namespace: cyber-ai-assurance (Zone F)

verification ×2 (egress: model-gateway only)

audit-observability ×2

notification ×2

namespace: cyber-ai-ingestion (async — Graph RAG feed)

connector workers (Confluence · GitHub · tickets)

chunk/enrich/entity-link jobs

outbox writers → RAG svc + Neo4j

🔒 NetworkPolicies enforce the invariants: **Zone A→B→C→D only, top-down** · only **cyber-ai-egress** may reach Neo4j / RAG service / warehouse / SIEM · **verification has no data-tier egress at all** · only **model-gateway** may reach the Enterprise LLM Gateway · all pods → **Kafka** (audit emit) allowed · **Redis** reachable from chat-api, tool-gateway, model-gateway/LLM shim only · **object storage** writable from egress + assurance zones only (evidence & drafts), read via evidence_store_tool.

On-prem data & messaging tier (outside the cluster; reachable only per NetworkPolicy)

Postgres HA cluster (on-prem)

platform state · Temporal history · schema-per-service

Neo4j cluster (on-prem, RO replica for queries)

entity spine · topology + knowledge edges

Kafka cluster (on-prem)

audit · tool calls · policy decisions · lifecycle · approvals · cost · ingest · query audit

Redis (on-prem, HA)

OpenSearch (on-prem)

Object storage (on-prem, S3-compatible)



5 • Logical View — six planes, enterprise-mapped

Left: the plane (from the design baseline). Right: what implements it in your enterprise.

L1 Experience

intake · streaming · status

OpenShift Route + chat-api · Entra OIDC sign-in at the edge

L2 Context

identity · tenant · session · memory

context-api + Entra ID · Entra groups → roles/entitlements · signed context token (5-min TTL)

L3 Control

intent · what exists · what's allowed

orchestrator · registry-api · policy-api on Postgres · fail closed both ways

L4 Execution

durable workflows · agents · sole egress

Temporal on OpenShift (history in Postgres) · agent-runtime · tool-gateway in the locked-down egress namespace

L5 Domain & Model

data products · graph · retrieval · LLM

cyber-domain-api → Neo4j (graph channel) + Enterprise RAG service (vector/lexical) + warehouse · model-gateway → **Enterprise LLM Gateway**

L6 Assurance

verify · audit · approvals

verification (no egress) · Kafka→audit · notification with Entra-group approver validation

6 · Sequence Diagram — one question, end to end

"Can CVE-2026-1234 on payments-gw reach customer data — and how do we fix it?" Every enterprise touchpoint shown in color.

- 1 **Analyst** → **chat-api**
Question + **Entra access token** (OIDC/PKCE sign-in already done in the browser).
- 2 **chat-api** → **Entra ID**
Validate token signature & claims against Entra JWKS; reject expired/invalid at the edge.
- 3 **chat-api** → **context-api**
Resolve identity → tenant, roles from **Entra groups**, entitlements, history.
Returns the platform's signed context token (5-min TTL, mTLS-bound) — the Entra token goes no further.
- 4 **orchestrator** → **registry + policy**
Intent = vulnerability chain analysis → approved workflow v1 → policy **ALLOW**: read-only, depth ≤ 4, PII masked, token budget set.
- 5 **workflow-runtime** → **Temporal**
Start durable execution; state persists in **Postgres**; progress streams to the analyst via Redis → SSE.
- 6 **agent-runtime** → **LLM Gateway**
Skills call **model-gateway** → **Enterprise LLM Gateway** for reasoning/extraction.
Registry-versioned prompts only; budgets metered per execution; usage → cost events.
- 7 **tool-gateway** → **Neo4j**
Graph-path query: guards clamp depth (e.g. 6→4), inject tenant scope, then execute **read-only**. Attack paths toward data stores returned with node/edge evidence.
- 8 **tool-gateway** → **RAG service** + **Neo4j**
Hybrid retrieval for remediation knowledge: **RAG service** (vector + lexical, entitlement-filtered pre-query) + **Neo4j graph expansion** (≤2 hops from payments-gw / CVE nodes). Fusion + rerank; every kept chunk → *ev://* evidence.
- 9 **workflow**
Deterministic risk scoring over the chains; agent drafts the answer + remediation plan, citing evidence per claim.
- 10 **verification**
Quality gate: evidence resolves, paths valid, masking intact.
FAIL → bounded improvement loop (max 2), executed by the workflow — verification itself touches nothing.
- 11 **chat-api** → **Analyst**
Final answer streamed with evidence refs and confidence.
If a write action were requested: park on Temporal signal → **Approver** (Entra-group-verified, no self-approval) → fresh write-class policy check at the gateway → deterministic write. (Deferred for now.)
- 12 **all services** → **Kafka**
Every step above emitted an audit event: decisions, tool calls with guards applied, query rewrites, token

usage, lifecycle.

Views: C4 L1-L3 · Deployment (OpenShift on-prem) · Logical planes · End-to-end sequence

Constraints honored: Entra ID auth · Enterprise LLM Gateway (sole model path) · Neo4j on-prem (graph) · RAG-as-a-Service (vector/lexical) · Postgres on-prem · Jira write path deferred

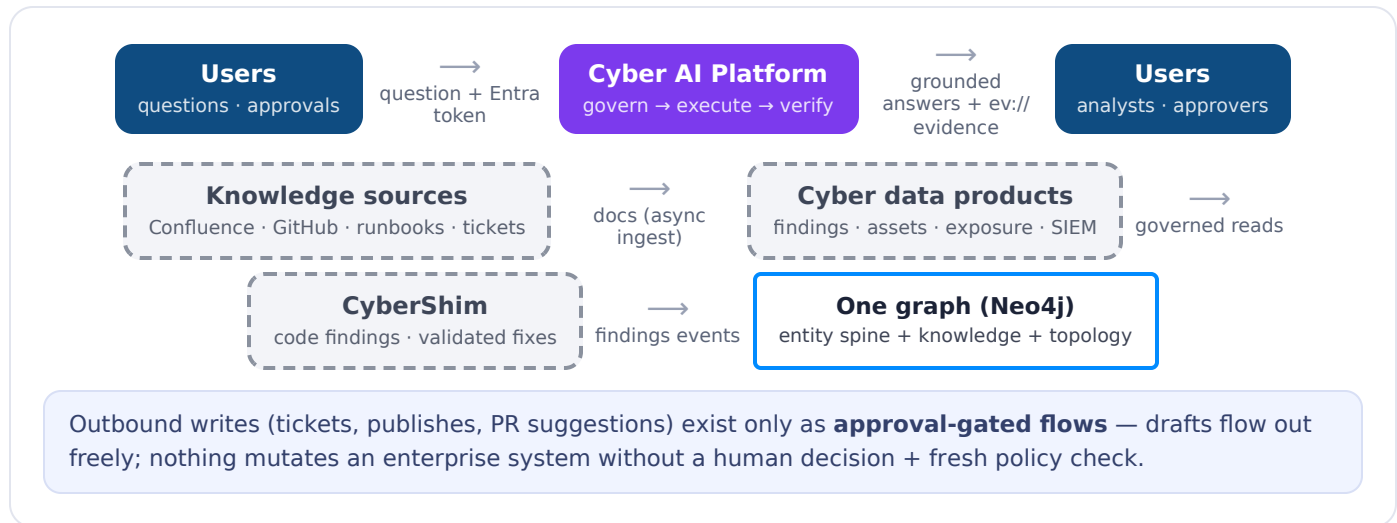
Derived from the Cyber AI Platform Consolidated Design Specification v1.0

2 · Data Flow Diagrams

How data moves through the platform. Solid nodes = platform-owned processing; white bordered = data stores; dashed = enterprise systems. Every flow that crosses into enterprise data passes the tool-gateway (red).

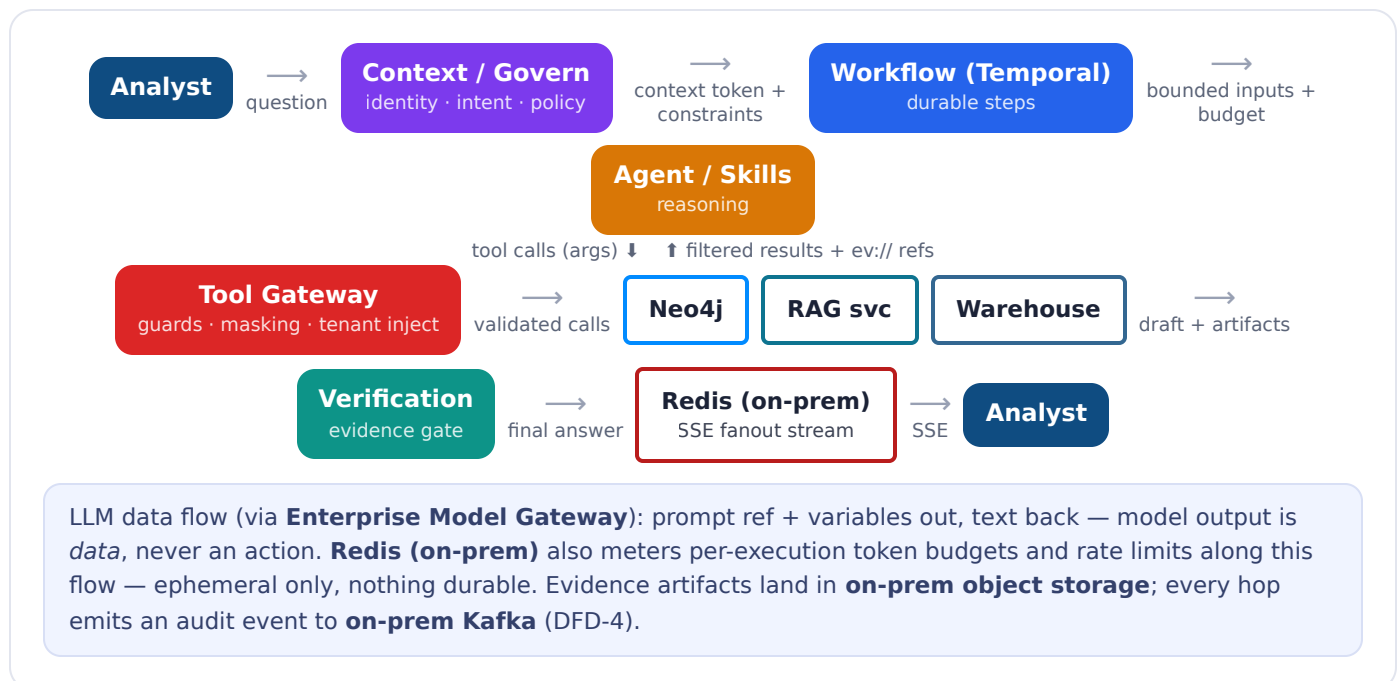
DFD-0 — Platform context

Top-level: what flows in, what flows out.



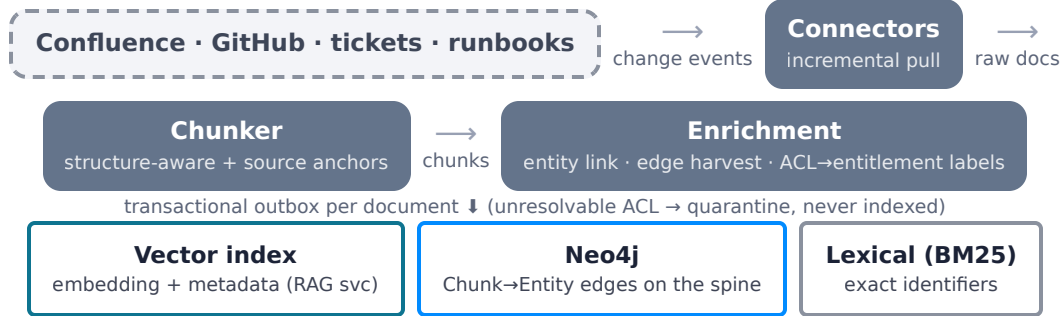
DFD-1 — Interactive request flow

A question becomes a grounded answer.



DFD-2 — Knowledge ingestion flow (async plane)

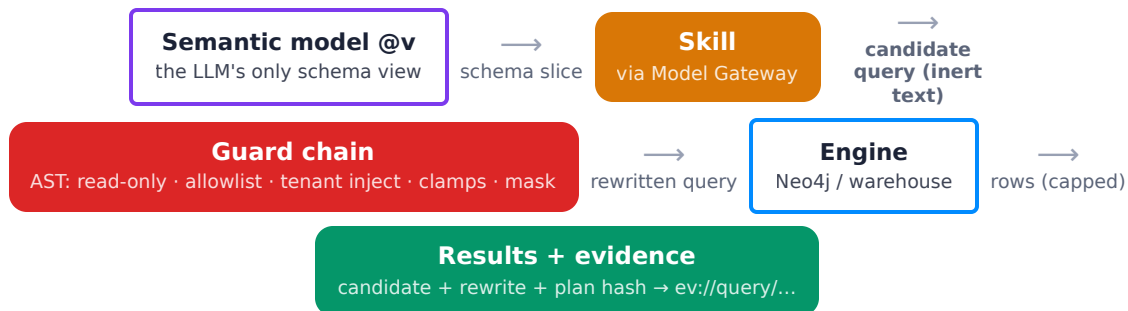
Sources become an entitlement-labeled, entity-linked triple index.



Superseded chunks are **tombstoned, not deleted** — past ev:// refs in audit records must always resolve.

DFD-3 — Governed NL2Query flow

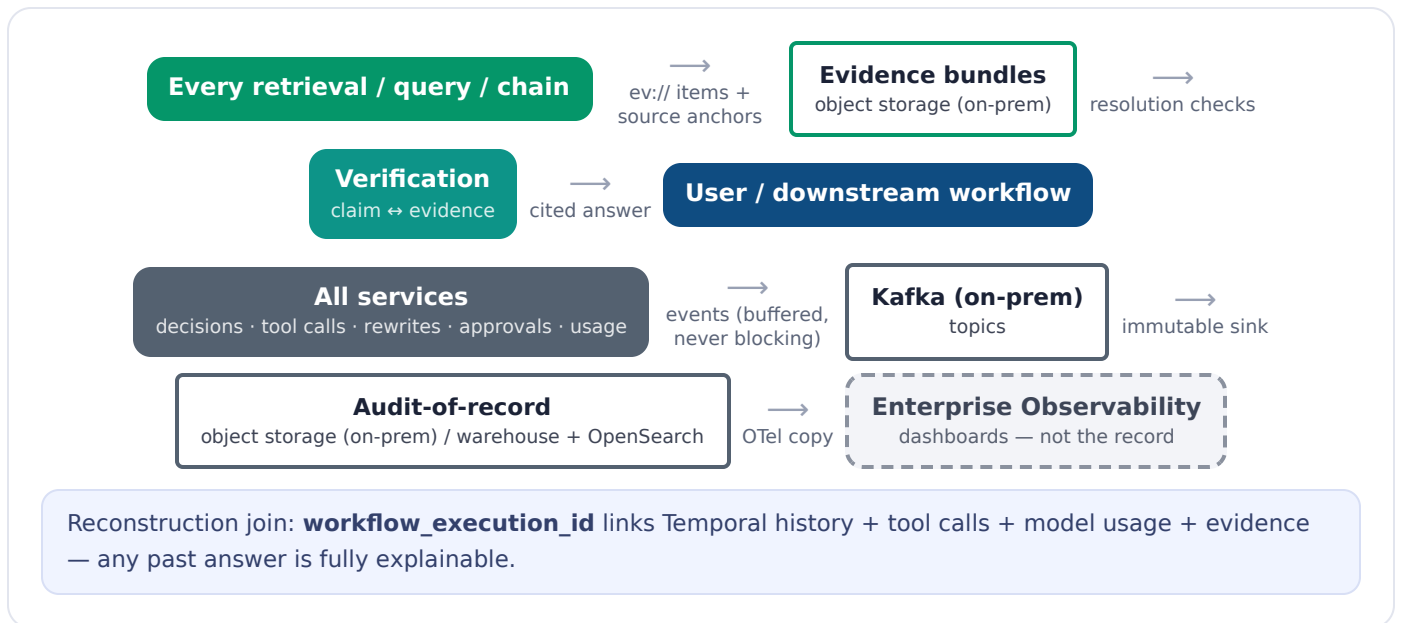
Natural language becomes an executed, audited query — with a rewrite boundary in the middle.



On GUARD_BLOCKED, **structured feedback** flows back to the skill (max 2 regenerations, loop owned by Temporal). The query itself becomes evidence — rewrites are visible, audited facts.

DFD-4 — Evidence & audit flow (cross-cutting)

Two record streams with different jobs: evidence proves answers; audit proves conduct.



DFD-5 — CyberShim findings flow into the spine

Code-level risk joins infrastructure risk in one graph.



Component & Sequence Diagrams — after the reuse decisions

Updated views reflecting the enterprise systems of record: **Enterprise Agent Registry** (agents · subagents · skills · tool access), **Enterprise Model Gateway**, and **Enterprise Observability** — alongside Entra ID, RAG-as-a-Service, Neo4j, Postgres, and OpenShift.

 **Build** — platform-owned, our code  **Thin adapter / shim** — small platform glue we build
 **Reuse** — enterprise shared service (dashed)

1 · Updated container view — what's ours vs. the enterprise's

Same six-plane shape; three former services are now dashed externals with a thin adapter left behind in the cluster.

ZONE A · EXPERIENCE

chat-api

intake · SSE · Entra token validation

Microsoft Entra ID

OIDC · groups → roles

ZONE B · CONTROL

context-api

token exchange · session · memory

orchestrator

intent → approved workflow

registry adapter

read-through cache · fail closed · concept mapping

Enterprise Agent Registry

agents · subagents · skills · tool access · versions

policy-api ★

allowed-now · constraints · obligations

ZONE C · EXECUTION HARNESS

workflow-runtime + Temporal ★

durable steps · approval parks · budgets

agent-runtime ★

bounded skill execution

LLM shim

prompt refs · per-execution budgets · cost events

Enterprise Model Gateway

routing · quotas · model safety

ZONE D · GUARDED EGRESS ★ (SOLE PATH TO DATA)

tool-gateway ★

guard chain · adapters

cyber-domain-api ★

graph · findings · knowledge · evidence

Neo4j (RO)

entity spine graph

Enterprise RAG Service

vector · lexical

CyberShim

code-security tool provider · scan (read) · findings (read) · fix (draft)

ZONE F · ASSURANCE

verification ★

quality gate · no egress

audit-of-record pipeline

Kafka → immutable sink (decisions · rewrites · approvals)

Enterprise Observability

OTel traces · metrics · logs · dashboards

notification

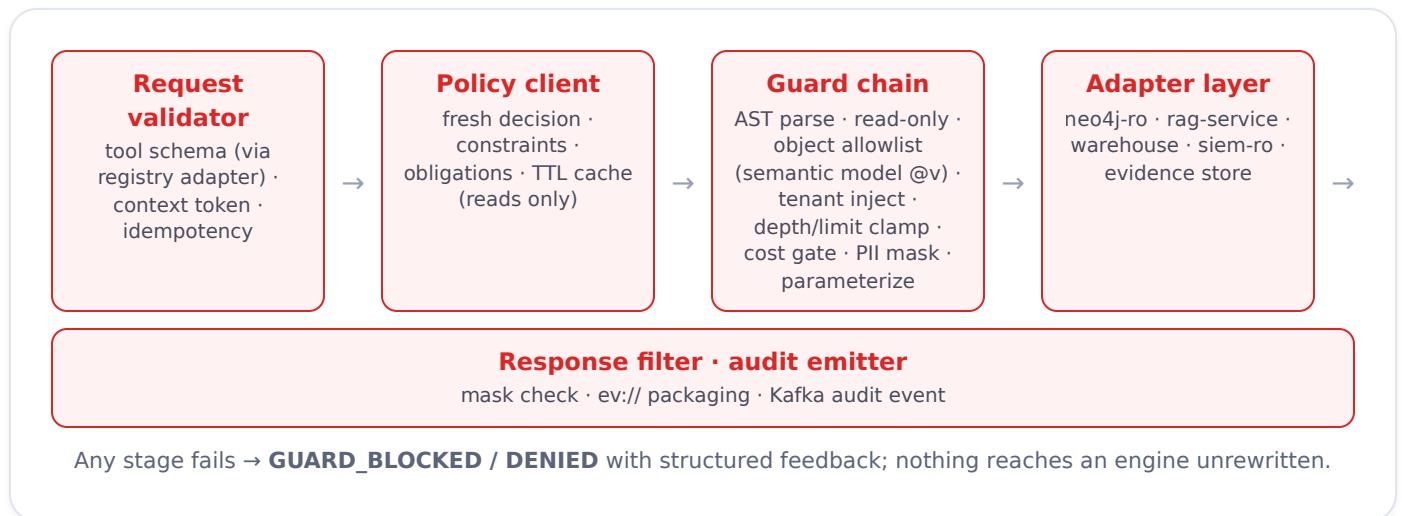
approvals via Entra groups

★ = the differentiated build focus · every solid box emits OTEL to Enterprise Observability; audit events additionally flow to the platform-owned immutable sink

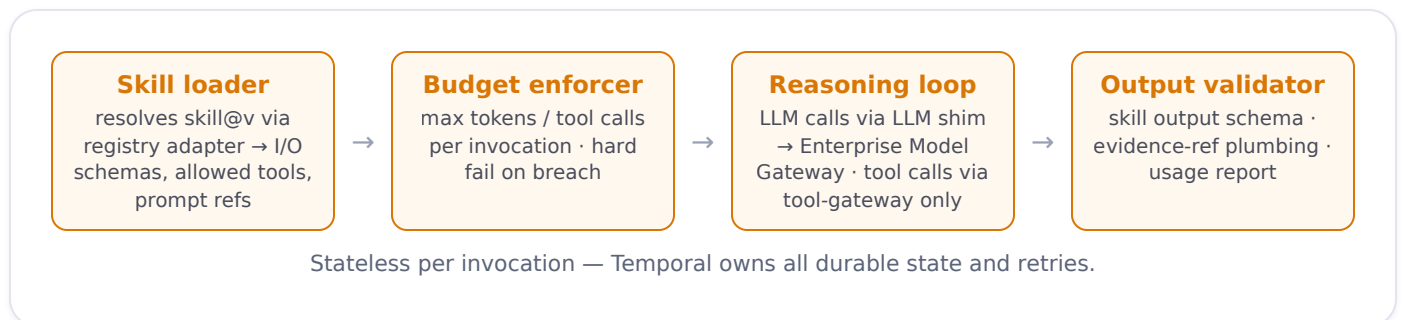
on-prem infrastructure (API-only access): **Postgres** · **Neo4j** · **Kafka** (event backbone: audit · tool calls · lifecycle · ingest) · **Redis** (SSE fanout · budgets · idempotency — ephemeral only) · **object storage** (evidence bundles · drafts · audit archives — write-once) · OpenSearch

2 · Component diagrams — inside the four build-critical containers

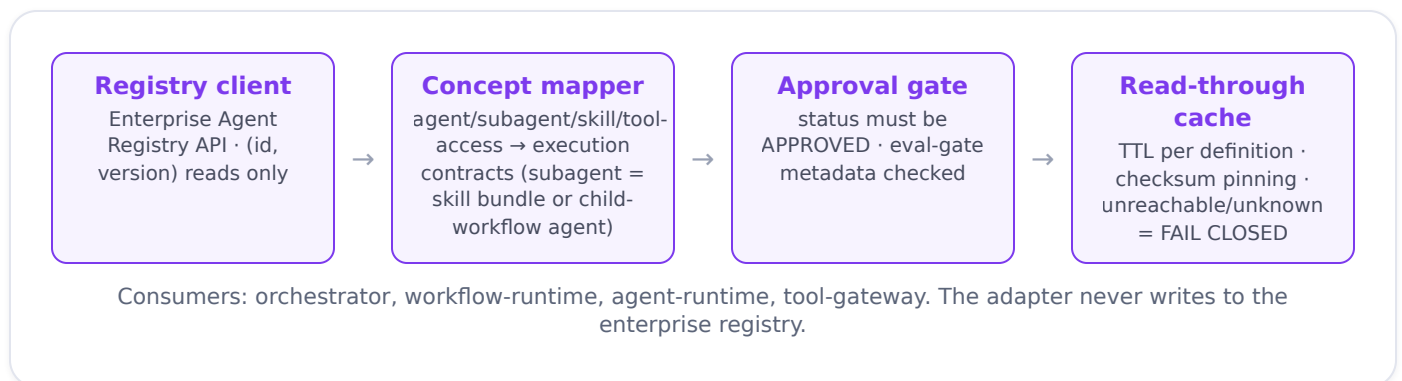
2.1 tool-gateway ★ — the guard pipeline



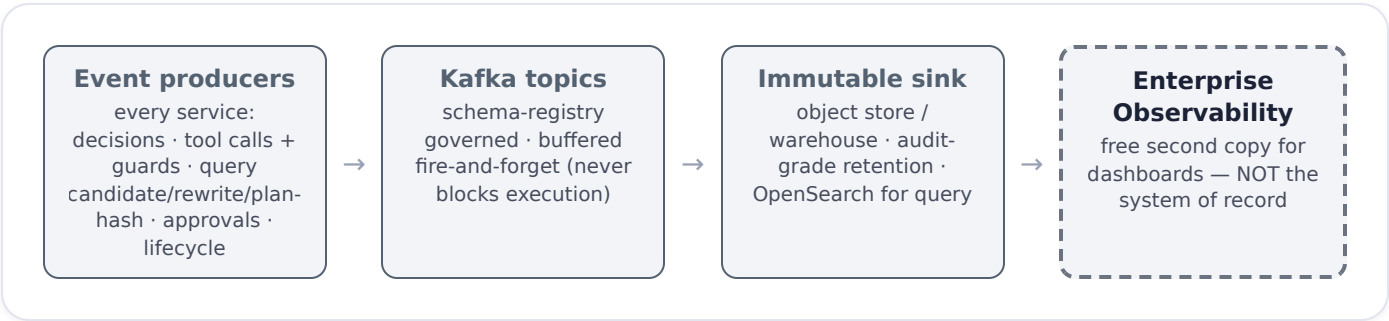
2.2 agent-runtime ★ — bounded reasoning



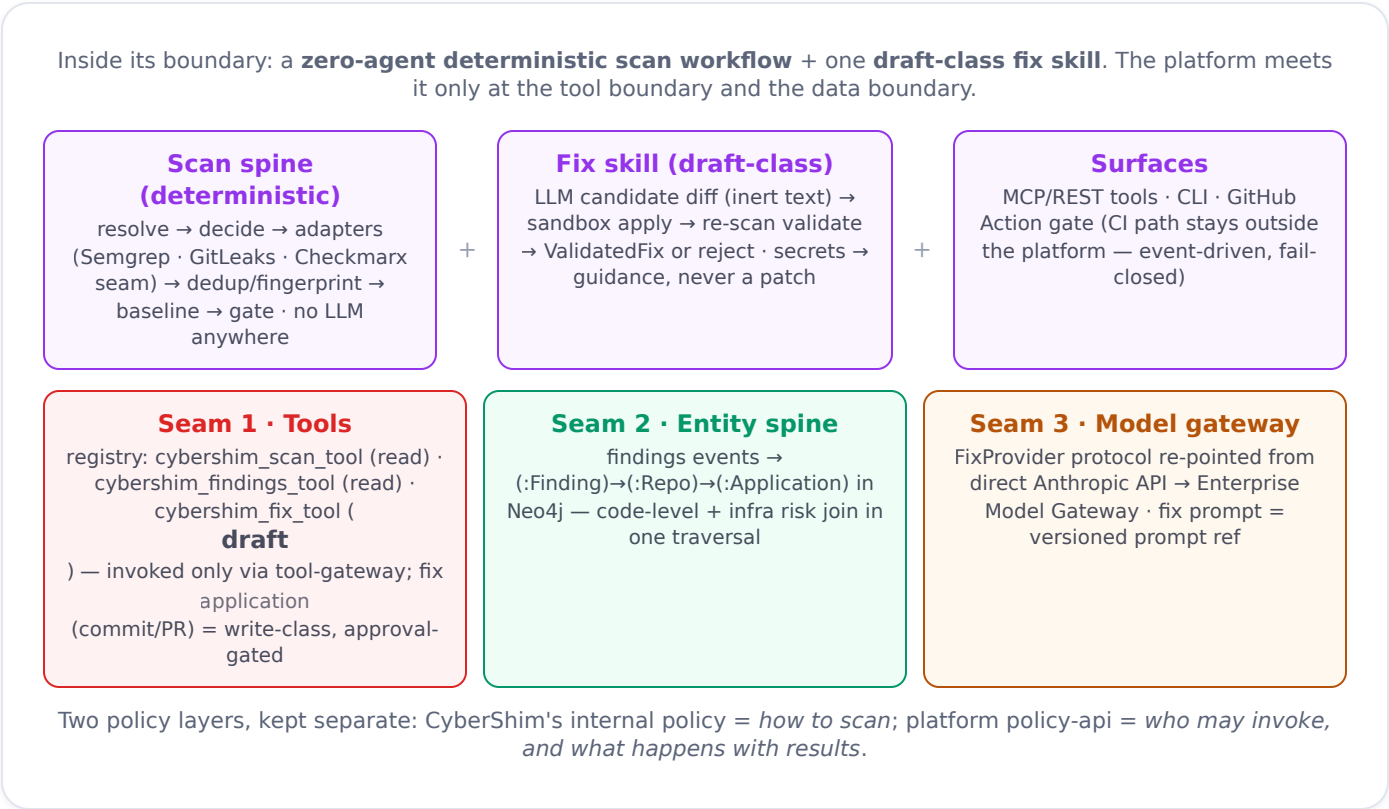
2.3 registry adapter — the enterprise-registry bridge



2.4 audit-of-record pipeline — compliance, not telemetry



2.5 CyberShim — governed tool provider (not an agent, not a platform workflow)



3 · Sequence diagrams

3.1 End-to-end question (updated with enterprise services)

"Can CVE-2026-1234 on payments-gw reach customer data?" — dashed chips are enterprise reuse.

1	Analyst → chat-api	Question + Entra access token; Entra ID JWKS validation at the edge.
2	chat-api → context-api	Token exchange → signed context token (5-min TTL); Entra groups → roles/entitlements.
3	orchestrator → registry adapter → Agent Registry	Resolve intent → fetch approved workflow + agent + skills + tool-access from the enterprise registry (cached, checksum-pinned). Unknown or not APPROVED → fail closed, nothing starts.
4	orchestrator → policy-api	ALLOW + constraints: read-only, depth ≤ 4, PII masked, execution token budget.
5	workflow-runtime → Temporal	Durable execution starts; progress → Redis → SSE to the analyst.
6	agent-runtime → LLM shim → Model Gateway	Reasoning/extraction calls: shim resolves <i>prompt://...@v</i> , meters the per-execution budget, then calls the enterprise model gateway . Usage → cost event tagged with workflow_execution_id.
7	tool-gateway → Neo4j RO	Graph-path query through the guard chain: depth clamp, tenant inject, read-only — attack paths + node/edge evidence.
8	tool-gateway → RAG Service + Neo4j	Hybrid retrieval: RAG service (vector+lexical, entitlement-filtered pre-query) + graph expansion ≤2 hops; fusion + rerank; chunks → ev:// evidence.
9	workflow → verification	Deterministic risk scoring → agent drafts answer with citations → quality gate (evidence resolves, paths valid, masking intact; ≤2 repair iterations).
10	chat-api → Analyst	Final grounded answer streamed. Throughout: OTel spans → Enterprise Observability ; audit events → platform immutable sink.

3.2 Governed NL2Query (guard-chain sequence)

"Which internet-facing servers have critical findings older than 90 days?"

- 1 **skill** → **registry adapter** Fetch **semantic model @v** — the LLM's only view of the schema (tables/labels, columns, synonyms, RLS predicate, exemplars).
- 2 **skill** → **LLM shim** → **Model Gateway** Generate **candidate query as inert text** + declared objects/intent.
- 3 **skill** → **tool-gateway** Invoke *sql_query_tool* / *cypher_query_tool* with the candidate text.
- 4 **guard chain** Parse to AST → read-only check → every object on the semantic-model allowlist → **rewrite**: inject tenant/RLS predicate, clamp depth 6→4, inject LIMIT, mask PII columns, parameterize literals → cost gate via query plan.
- 5 **adapter** → **Neo4j** / **Warehouse** Execute the **rewritten** query only (engine chosen by declared query class — never by the LLM); statement timeout + row cap.
- 6 **tool-gateway** → **skill** Results + *guards_applied* + rewritten query + plan hash → packaged as *ev://query/...* evidence. Skill summarizes with the query attached as proof.

On **GUARD_BLOCKED**: structured feedback (which guard, why) returns to the workflow → bounded regeneration (max 2) with the feedback in the prompt → then fail the step with a user-safe message. The repair loop is owned by Temporal, not the skill.

3.3 CyberShim from the platform — "fix this finding, following our runbook"

A future `code_security_agent` composing CyberShim tools with Graph RAG — CyberShim itself unchanged.

- 1 **Developer** → **platform** "What security findings are open on payments-api, and how do I fix the SQLi one?"
- 2 **governance** Intent → approved code-security workflow; policy: CyberShim tools read/draft only, fix application approval-gated.
- 3 **tool-gateway** → **CyberShim** *cybershim_findings_tool*: open findings for the repo (tenant-scoped, read-class).
- 4 **tool-gateway** → **Neo4j** + **RAG svc** Graph RAG: secure-coding runbook + exception policy for the app, found via the Finding→Repo→Application spine neighborhood.
- 5 **tool-gateway** → **CyberShim** *cybershim_fix_tool*: CyberShim generates the candidate diff (LLM via **Model Gateway**), sandbox-applies, re-scans — returns a **validated diff as a draft**, or rotation guidance for secrets.
- 6 **agent** → **verification** Answer assembled: findings + validated diff + runbook citations, every claim evidence-backed; quality gate passes.
- 7 **Approver / Dev** Applying the diff (commit / PR) is **write-class**: platform-initiated → approval park + fresh policy check; CI path → developer applies it themselves. Either way, a human is in the loop before code changes.

Build vs Reuse views · Enterprise Agent Registry / Model Gateway / Observability as systems of record
★ build focus: tool-gateway guard chain · policy-api · Temporal harness · agent-runtime · Graph RAG value layer · verification

Derived from the Cyber AI Platform Consolidated Design Specification v1.0 + enterprise alignment updates

How each cyber use case works

Every use case follows the **same simple rhythm**: ask → check it's allowed → run governed steps → gather evidence → verify → answer (and, only if a human approves, act). The colored labels below show *who does each step*.

WHO'S WHO

User

Governance (intent · registry · policy)

Workflow (Temporal — owns the steps)

Agent / LLM (thinks, never touches data)

Tool Gateway (guards every call)

Data plane (graph · warehouse · vector · SIEM)

Verification (quality gate, no egress)

Human approver

The shared spine (every use case starts the same way): User asks → Governance figures out the intent, finds the *approved* workflow for it, and Policy says "allowed, with these limits" → the Workflow runs its fixed steps. Agents fill in the thinking inside each step; the Gateway executes every data touch. Nothing below differs in this part — only the middle steps change.

1 Vulnerability Chain Analysis MVP · live first

"Can CVE-2026-1234 on payments-gw reach customer data?"

- 1 **USER** asks the question in chat.
- 2 **GOVERNANCE** recognizes the intent → picks the approved *vulnerability chaining* workflow → policy allows it read-only, graph depth ≤ 4 , PII masked.
- 3 **WORKFLOW** resolves the entities: which CVE, which application, which assets.
- 4 **DATA PLANE** returns the findings + CVE enrichment (CVSS, EPSS, known-exploited) via the Gateway.
- 5 **GATEWAY** runs three lookups *in parallel*: network exposure, **graph paths from the asset toward data stores**, and data sensitivity of anything reachable.
The graph query is depth-clamped and tenant-scoped by the guards — automatically.
- 6 **WORKFLOW** scores each chain with a **deterministic scorer** (versioned function — the LLM never invents the score).
- 7 **AGENT** drafts the explanation + remediation plan, citing evidence refs for every claim.
- 8 **VERIFICATION** checks: do all evidence refs resolve? Are the paths valid? Is anything unmasked? Fail → one bounded retry.

Result: "Yes — 2 attack paths reach the customer DB. Risk 87/100. Here's the evidence and the remediation plan." Streamed to the user with clickable evidence.

2 Knowledge Q&A (Graph RAG)

runbooks · Confluence · past tickets

"How do we remediate CVE-2026-1234 on payments-gw?"

- 1 **GOVERNANCE** routes to the knowledge-retrieval workflow; policy sets max 2 graph hops, 40 chunks, entitlement filter **required**.
- 2 **WORKFLOW** links the question to spine entities: *app:payments-gw*, *cve:CVE-2026-1234*.
- 3 **GATEWAY** makes **one hybrid retrieval call** — three channels at once:
 - **Vector** — meaning-similar chunks (filtered to what *this user* may see, before search)
 - **Lexical** — exact matches on "CVE-2026-1234", hostnames
 - **Graph** — chunks *connected to* those entities: the asset-class runbook, the app's exception policy, the past ticket for the same CVE elsewhere
- 4 **WORKFLOW** fuses and reranks the candidates (deterministic fusion; rerank reorders, never adds) and packages each kept chunk as evidence with its source anchor.
- 5 **AGENT** writes the answer — every claim must cite an *ev://* reference.
- 6 **VERIFICATION** fails any claim whose evidence doesn't resolve.

Result: A grounded answer citing the runbook section, policy page, and prior ticket — including documents that share *no words* with the question but sit in its graph neighborhood.

3 Ask-the-Data (NL2SQL / NL2Cypher) governed query pattern

"Which internet-facing servers have critical findings older than 90 days, by app owner?"

- 1 **GOVERNANCE** binds the request to **one approved semantic model version** — the only "schema" the LLM ever sees.
- 2 **AGENT** drafts a *candidate* SQL/Cypher query — as inert text. It cannot run anything.
- 3 **GATEWAY** parses the text into an AST and runs the guard chain: read-only? objects on the allowlist? Then it **rewrites** the query — injects the tenant filter, clamps depth/limits, masks PII columns, checks the cost estimate.
The LLM's version never executes. Only the rewritten query does — and the rewrite is recorded (e.g. "depth 6→4").
- 4 **DATA PLANE** executes it on the right engine (warehouse for tables, graph engine for traversals — routing is fixed, not the LLM's choice).
- 5 **WORKFLOW** keeps the query itself as evidence: candidate + rewritten form + plan hash. If a guard blocked it, the agent gets the feedback and retries — max 2 times.
- 6 **AGENT** summarizes the result table into the answer, with the query attached as proof.

🛡️ Why it's safe: wrong table → blocked. Write attempt → blocked. Missing tenant filter → injected automatically. Expensive scan → blocked before it runs.

Result: The answer + the exact audited query that produced it. Adding a new data domain later = registering a new semantic model, zero new code.

4 Investigation / Alert Triage

SIEM integration · future horizon

"Triage this alert: suspicious auth on srv-01."

- 1 **GOVERNANCE** routes to the triage workflow. Policy: SIEM is **read-only**; containment actions (isolate host, disable account) are **hard-denied**.
- 2 **GATEWAY** pulls alert context from the SIEM through the new read-only *siem_query_tool* — same guard pipeline as everything else.
- 3 **DATA PLANE** finds **similar past incidents** (vector search over the incident corpus — just another ingested source).
- 4 **GATEWAY** runs a **blast-radius graph query**: from srv-01, what can be reached? Reuses the exact same topology graph as use case 1.
- 5 **WORKFLOW** can even call *vulnerability chaining* as a **child workflow**: "is this asset on a known attack path to sensitive data?" (child re-passes the full governance gate).
- 6 **WORKFLOW** computes triage priority with a deterministic scorer over all of it.
- 7 **AGENT** drafts the triage recommendation + evidence; **VERIFICATION** gates it.
- 8 **HUMAN** approves before anything is written — and even then, only case annotations / escalation tickets. Never containment.

Result: "P1 — similar to incident INC-2210, asset sits 2 hops from restricted data, exploit is in the wild. Recommend escalation." Analyst decides; platform never acts alone.

5 Threat Modeling future horizon

"Threat-model the new payments checkout service."

- 1 **GOVERNANCE** routes to the threat-model workflow.
- 2 **GATEWAY** retrieves the design docs via **Graph RAG** (use case 2's machinery, different profile) — architecture pages, data-flow diagrams, prior reviews.
- 3 **DATA PLANE** supplies the real topology from the graph: what the service actually connects to, which data stores, which identities.
- 4 **AGENT** maps trust boundaries and enumerates STRIDE threats *per boundary* — each threat tied to real nodes/edges, with evidence refs.
- 5 **WORKFLOW** checks control coverage against the control catalog and scores gaps **deterministically**.
- 6 **VERIFICATION** gates the draft; **HUMAN** approves before the threat-model doc is published or gap tickets are raised.

Result: A threat model grounded in the *actual* deployed topology, not just the diagram — with a scored gap list ready for review.

6 Control Mapping / Compliance

recommended 2nd agent · read-heavy

"How do our controls map to NIST CSF, and where are the gaps?"

- 1 **GOVERNANCE** routes to the control-mapping workflow (lowest write risk — that's why it goes second).
- 2 **GATEWAY** pulls the internal control catalog + framework definitions; **Graph RAG** retrieves evidence of each control in practice (policies, runbooks, past tickets).
- 3 **AGENT** proposes the crosswalk: internal control ↔ framework requirement, each pairing evidence-backed.
- 4 **WORKFLOW** runs deterministic gap analysis: requirements with no mapped control, controls with no evidence.
- 5 **VERIFICATION** voids any mapping whose evidence doesn't resolve; **HUMAN** approves the published crosswalk.

Result: An evidence-backed crosswalk + gap list. Proves the onboarding model: this whole agent shipped as **config + registry entries**, zero platform code.

7 Composed Review (multi-agent)

workflows compose — agents don't chat

"Give me a full cyber risk review of the payments portfolio."

- 1 **GOVERNANCE** approves the **parent** workflow with one aggregate budget.
- 2 **PARENT WORKFLOW** launches children in parallel — vulnerability chaining, vendor exposure, control gaps. **Each child re-passes the full governance gate**; the parent's authority is not inherited.
- 3 **CHILDREN** return **typed reports** (e.g. *VulnChainReport@1*) — structured artifacts with evidence refs, never prose passed between models.
- 4 **PARENT** joins and ranks the portfolio with **deterministic logic** — this parent has no LLM of its own.
- 5 **VERIFICATION** gates the combined report; **HUMAN** approves publication.

Result: One ranked risk review built from three governed use cases — every number traceable through a child's evidence to a source system.

Same spine every time: **ask** → **govern** → **run fixed steps** → **evidence** → **verify** → **(approve)** → **answer**.

The LLM only ever thinks and drafts. Deterministic code validates, executes, scores, and records.

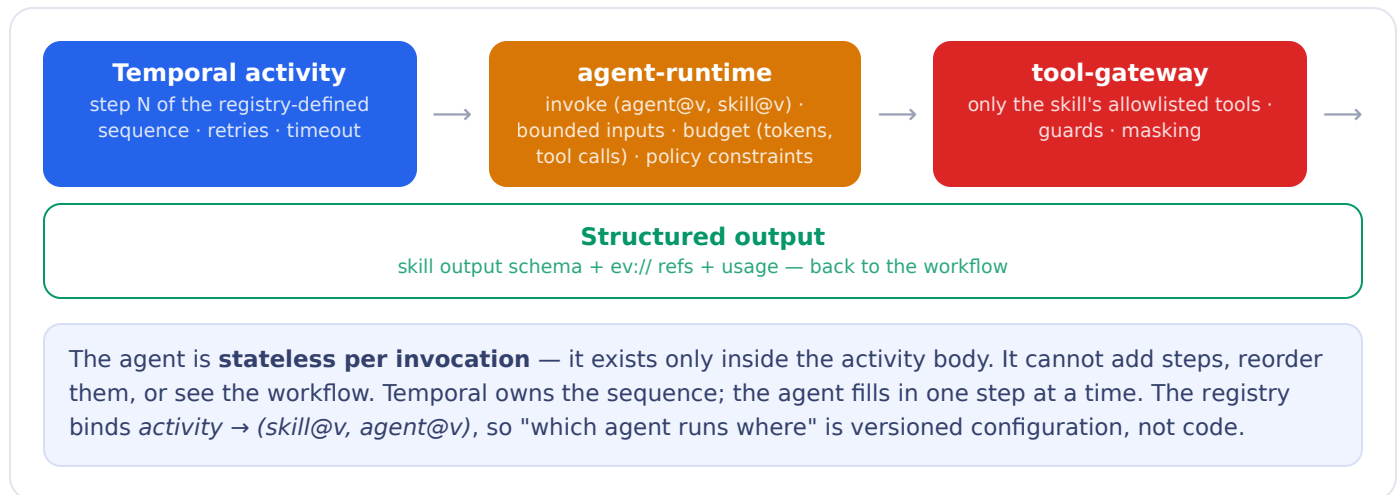
Cyber AI Platform · derived from Consolidated Design Specification v1.0 · Jira write path deferred

5 · Workflow Expansion — where each agent is called

The "Workflow (Temporal)" box from the sequence diagrams, opened up. Each activity is a logical box showing the **skill** it runs and the **agent** bound to it. Amber = agent-bound (LLM reasoning inside), blue = deterministic (no agent), teal = verification loop, pink = human approval park.

-  **Agent-bound activity** — agent-runtime invoked with (agent@v, skill@v, budget)
-  **Deterministic activity** — pure code, no agent, no LLM
-  **Quality loop**
-  **Approval park (signal)**

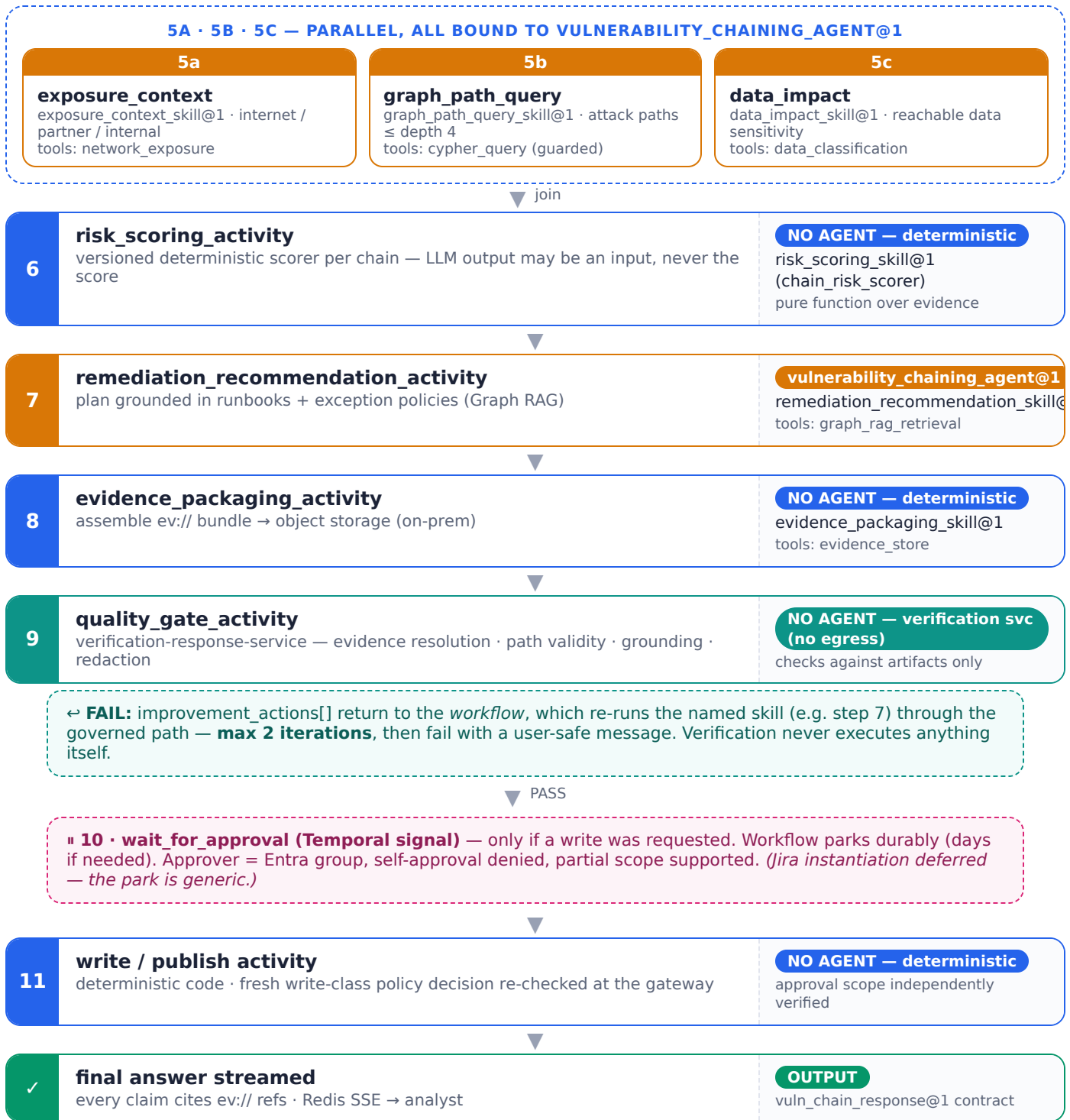
How an activity calls an agent (the invocation contract)



vulnerability_chaining_workflow@1 — expanded

One parent agent (vulnerability_chaining_agent@1) bound to the reasoning activities; scoring, evidence, and writes are deterministic. Jira draft/create steps deferred.





Where the agent is called, summarized: steps 1-5, 7 invoke **vulnerability_chaining_agent@1** (one bounded invocation per activity, different skill each time). Steps 6, 8, 11 are pure code. Step 9 is the verification service. The agent never appears between steps — only inside them.

Multi-agent workflow — alert_triage_workflow (P4) expanded

Same pattern, but activities bind to different agents — including the shared knowledge_retrieval_agent. "Which agent" is per-activity registry configuration.



▼ fan-out

PARALLEL — THREE DIFFERENT BINDINGS

2a

similar_incidents

detection_triage_agent@1 ·
similar_incident_retrieval_skill
tools: incident_store (vector)

2b

blast_radius

detection_triage_agent@1 ·
blast_radius_skill
tools: cypher_query (graph)

2c

runbook_context

knowledge_retrieval_agent@1
(shared) · knowledge_retrieval_skill
tools: graph_rag_retrieval

3

child: vulnerability_chaining_workflow

"is this asset on a known attack path?" — child re-passes the full governance gate; returns VulnChainReport@1

CHILD WORKFLOW

parent authority NOT inherited ·
budget composed

4

trriage_priority_scoring

deterministic scorer over all inputs

NO AGENT — deterministic

trriage_priority_scorer@1

5

trriage_recommendation_activity

evidence-cited recommendation draft

detection_triage_agent@1

trriage_recommendation_skill@1

▼ verify (loop ≤2) → approval park → deterministic write (case annotation / escalation only — containment hard-denied)

✓

TriageRecommendation@1

typed output contract with evidence refs

OUTPUT

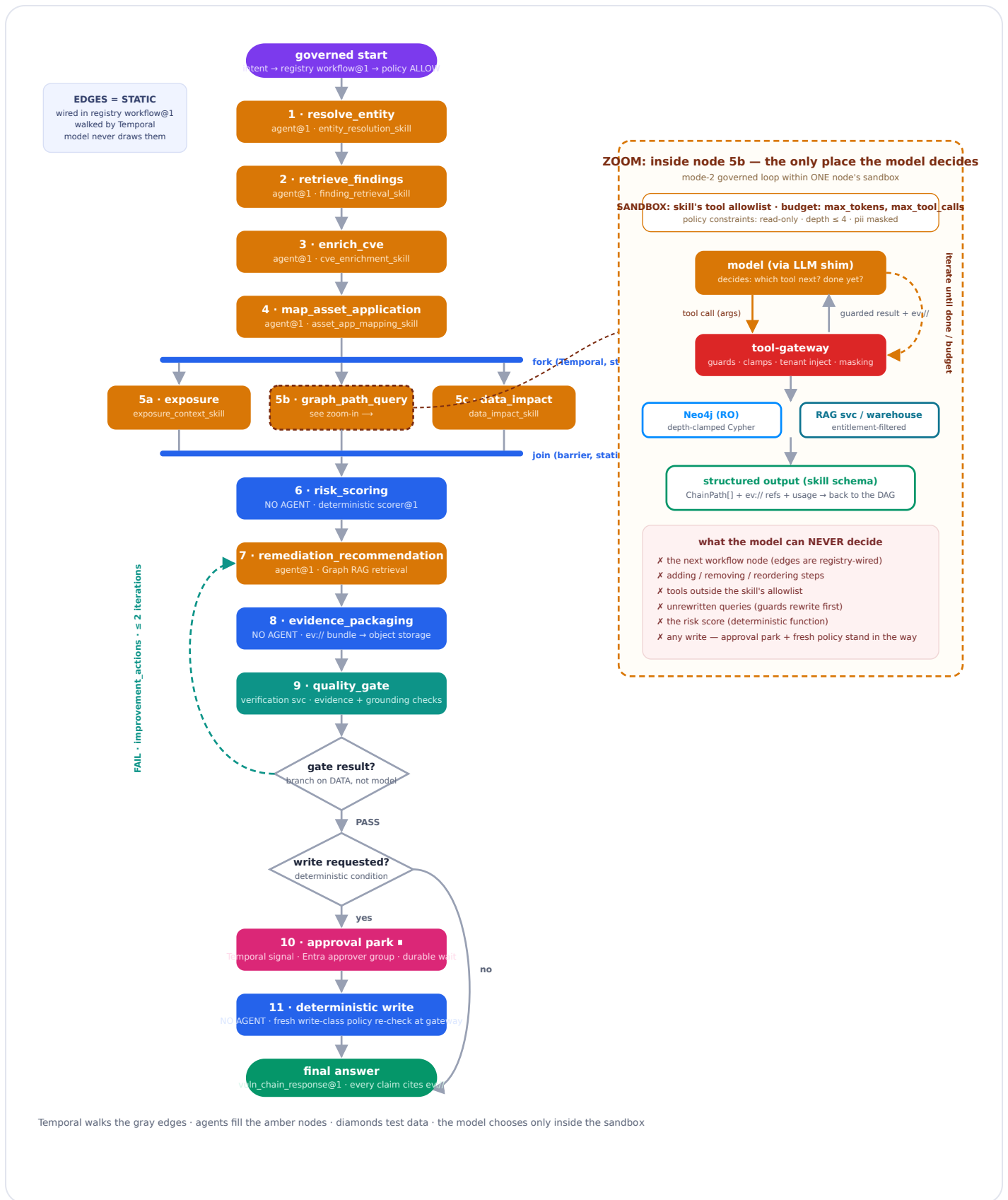
The three binding patterns visible here: a workflow's activities can bind to **its own agent** (trriage steps), a **shared agent** (knowledge_retrieval_agent@1, reused across threat modeling, control mapping, triage), or **no agent at all** (scoring, child-workflow join). One workflow, many agents; one agent, many workflows — the registry mapping *activity* → (*skill@v*, *agent@v*) is the single source of truth for "who runs where."

5b · The workflow as a graph — who draws the edges, who fills the nodes

Gray edges are wired at design time (registry definition, walked by Temporal — the model never draws them). **Amber nodes are where the model works** — and the zoom-in on the right shows the only place the model "decides": which allowlisted tool to call next, *inside* a node, under a budget. Branch diamonds are deterministic code testing data, never model choice.

 agent-bound node (model fills it)  deterministic node (pure code)  verification (no egress)

 human approval park  deterministic branch (on data)  static edge — registry-wired, Temporal-walked



Framework decision (recorded): Temporal remains the sole walker of workflow edges — no agent framework at the orchestration layer. **LangGraph** is approved as an optional *in-activity* skill library under three constraints: (1) no LangGraph checkpointing/persistence — Temporal owns durability, in-node graphs are stateless per invocation; (2) tools bind only to the tool-gateway (MCP facade) — no LangChain community connectors; (3) the LLM client is the LLM shim (prompt refs + per-execution budgets). **Google ADK rejected:**

GCP/Vertex deployment coupling and Gemini gravity are incompatible with on-prem OpenShift + Entra + the model-agnostic enterprise LLM gateway. Claude Agent SDK remains the candidate loop engine for mode-2 tool loops.

6 · MVP — the minimal build

Smallest thing that proves the architecture: **one governed, evidence-backed answer to "can CVE-X on app-Y reach sensitive data, and how do we fix it?" — read-only, end-to-end.**

Because the MVP performs no enterprise writes, the entire approval/notification machinery drops out of scope, which is the single biggest simplifier.

6.1 Minimal service set (10 deployables + 1 library)

#	COMPONENT	MVP-MINIMAL FORM
1	chat-api	Entra token validation · POST /messages/stream · SSE via Redis · status endpoint. No conversation UI beyond basic history.
2	context-api	Token exchange (Entra → signed 5-min context token) · session + conversation rows in Postgres. No memory facts yet.
3	orchestrator	Rule-based intent (pattern + entity match — no LLM needed for one intent) · routes to the one approved workflow or declines.
4	registry adapter	Read-through client on the Enterprise Agent Registry, (id,version) pinning, APPROVED check, fail closed. Library, not a service, if the registry API is stable.
5	policy-api	One policy bundle (mvp_vuln_chain_policy_v1): allow the workflow + 8 read tools; constraints (read_only, depth≤4, pii masked, budgets); deny-all else. Static rules — no policy authoring UI.
6	workflow-runtime + Temporal	One workflow definition, signals for cancel only (no approvals), step history, quality-loop bound = 2.
7	agent-runtime	Skill execution with I/O schema validation + budget hard-stop. Mode 1 only — fixed skill per activity, no tool loops.
8	LLM shim	Library inside agent-runtime/verification: prompt-ref resolution, per-execution budget in Redis, cost events → Kafka.
9	tool-gateway	Schema validation · policy check · parameterized-template guards (see 6.3) · masking · audit emit. Full AST guard chain deferred with NL2Query.
10	cyber-domain-api	8 read endpoints backed by: Neo4j RO (paths), findings/asset/exposure/classification data products, object store (evidence).
11	verification-response	4 checks: evidence_resolution · chain_path_validity · risk_explanation · redaction. No egress. improvement_actions loop ≤2.

✓ **Invariants that survive minimalization untouched:** single egress (NetworkPolicy from day 1, even with 3 namespaces: edge / core / egress) · fail closed on unknown/unapproved/undecided · LLM never writes · signed context token on every hop · every claim carries a resolvable ev:// ref · every call emits an audit event. These are the architecture — cut features, never these.

6.2 Minimal registry content (filed in the Enterprise Agent Registry)

Definitions

- **1 agent** — vulnerability_chaining_agent@1
- **1 workflow** — vulnerability_chaining_workflow@1 (steps 1-9 of the baseline §5 flowchart; park + write steps absent)
- **9 skills** — entity_resolution · finding_retrieval · cve_enrichment · asset_app_mapping · exposure_context · graph_path_query · data_impact · remediation_recommendation (runbook stub) · evidence_packaging + **1 deterministic scorer** (chain_risk_scorer@1)
- **8 read tools** — graph_query (templated) · vulnerability_findings · cve_enrichment · asset_inventory · application_ownership · network_exposure · data_classification · evidence_store

Governance artifacts

- **1 policy bundle** — allow/constraints/obligations as above
- **1 output contract** — vuln_chain_response@1
- **~8 prompt refs** — one per LLM-bearing skill, versioned
- **1 eval set** — 25-30 golden questions with expected evidence shape; gates APPROVED status

6.3 Minimal guard posture (before the full AST chain)

MVP ships **templated queries, not NL2Query**: graph_path_query uses a registry-stored parameterized Cypher template — the LLM supplies only *parameter values* (start nodes, target labels), never query text. The gateway validates parameters against the tool schema, clamps depth to the policy bound, injects the tenant predicate into the template, and masks per obligations. This delivers the security properties of guards g2-g5/g7-g8 with ~10% of the build, and the full AST chain (g1-g8) arrives with NL2Query in the next phase without changing the tool contract.

6.4 Minimal infrastructure

INFRA

MVP-MINIMAL FORM

OpenShift	3 namespaces (edge / core / egress) with the top-down NetworkPolicies; consolidate to zones A-F later without re-architecture
Postgres (on-prem)	1 HA instance, schemas: context, policy, workflow_meta + Temporal history
Neo4j (on-prem)	RO replica access only; topology graph as provided by the graph team — no knowledge edges yet
Redis (on-prem)	SSE fanout + budgets + idempotency
Kafka (on-prem)	2 topics: audit.events.v1, tool.calls.v1 → object-storage sink (add the other six topics as features land)
Object storage	evidence bundles + audit archive buckets, write-once
Enterprise services	Entra ID (OIDC + groups) · Enterprise LLM Gateway (via shim) · Enterprise Agent Registry · Enterprise Observability (OTel)

6.5 Explicitly deferred (with the seam that makes each additive)

Cut from MVP — nothing here blocks the demo: approvals/notification service (no writes exist) · Jira path (deferred throughout) · Graph RAG ingestion plane & knowledge edges (remediation uses a curated runbook-lookup stub; seam = knowledge_retrieval tool contract) · NL2Query + semantic models + AST guard chain (seam = same tool contract, templates → generated queries) · Graph RAG hybrid retrieval · memory facts · CyberShim integration (seam = 3 tool registrations) · second agent & composition (seam = registry bundles) · mode-2 loops / LangGraph / Agent SDK (seam = agent-runtime invocation contract) · OpenSearch · MCP facade · Entra Agent ID delegation chains (MVP: workload identity + user token) · A2A · dashboards beyond enterprise-observability defaults.

6.6 Definition of done (the MVP demo, run twice unattended)

- Analyst signs in via Entra, asks: "*Can CVE-2026-1234 on payments-gw reach customer data?*" → streamed progress → grounded answer with risk score, chain paths, remediation, **every claim carrying a clickable ev:// ref that resolves.**
- Same question from a user in another tenant / without entitlements → different (filtered) or declined result — **tenant-boundary test green.**
- Out-of-scope question ("patch the server") → orchestrator declines; policy denies; audit shows the denial.
- Kill a pod mid-run → Temporal resumes; answer still arrives; no duplicate tool calls (idempotency).
- Depth-6 path request → gateway clamps to 4; the clamp is visible in the tool.calls audit event.
- Verification forced to FAIL (seeded bad draft) → one repair iteration → PASS or user-safe failure.
- Eval set $\geq$ threshold; registry entry APPROVED; every answer reconstructable by workflow_execution_id join.